

One Canvas: Five Runtimes and Purposeful Mixing

The builder in RosettaDash is written in Angular with NestJS handling the API work. When you export, the same layout can leave as React, Angular, Vue, Svelte, or W3C custom elements. You choose the target at export time; the canvas does not change. The first article introduced that idea. This article shows how it works in the repo: the five Destination Atlas proofs, and the one proof (Svelte) that deliberately embeds another framework in some of its screens.

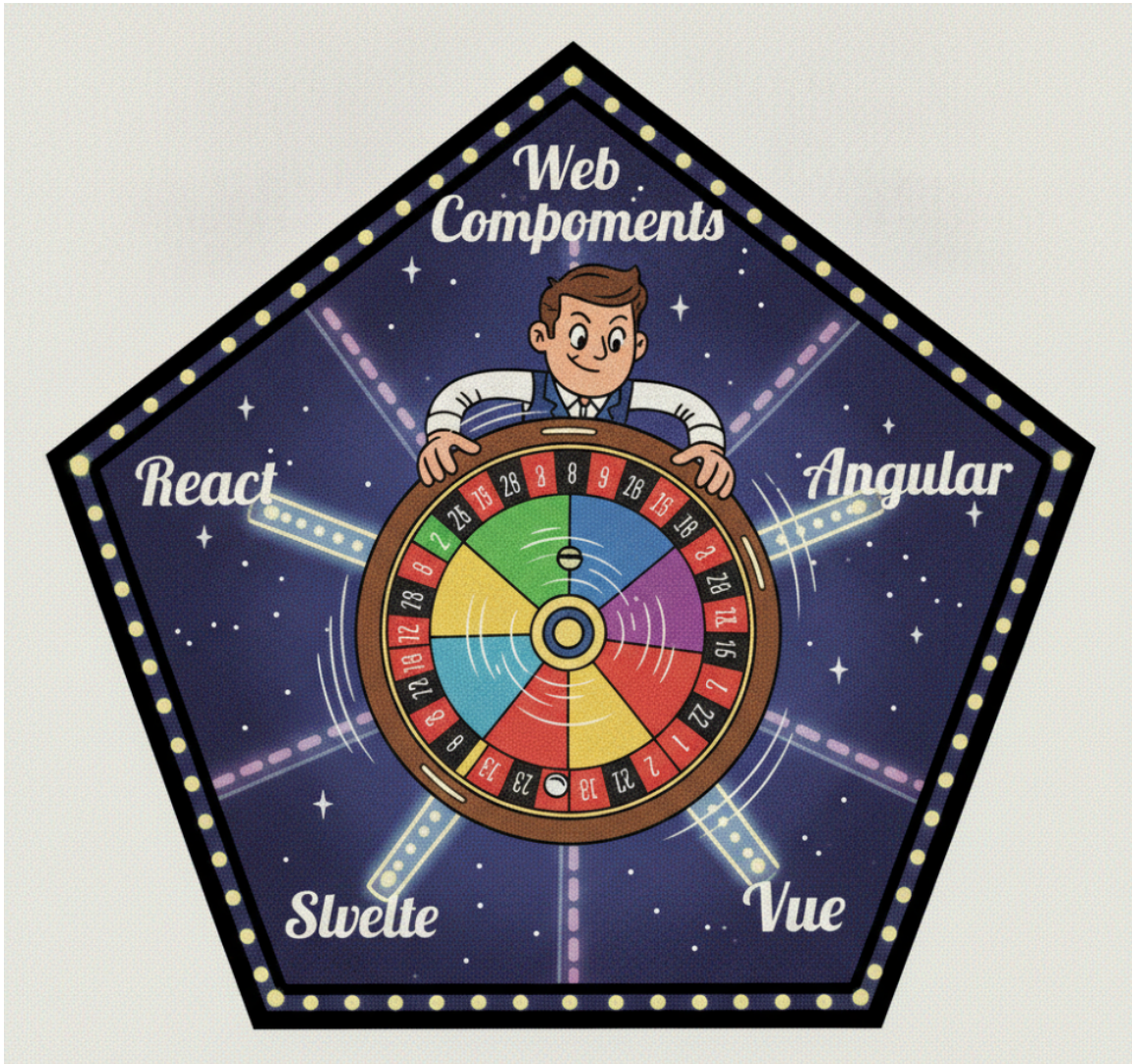


Figure 1: Choose a framework and take a spin.

One foundation, five implementations

Compose on the canvas, fix validation errors, then export. Before any framework-specific files are written, the builder turns your graph into one validated description: which com-

ponents you placed, how they connect, which env vars and server choices apply, and which UI target you picked. Every exporter reads that same description. Adding another framework does not mean rebuilding the builder — it means adding another exporter that understands the same input.

The default download is standalone source. Drop the zip into a project, set environment variables, and run. You do not need to install `@rosettadash/*` to ship what you just exported. If you prefer npm instead, install the packages for the framework you already use and import a typed component.

```
npm install @rosettadash/vue
# or: @rosettadash/react |
#     @rosettadash/angular |
#     @rosettadash/svelte |
#     @rosettadash/web-components
```

Everything runs on your machine. The generated files live in your tree.

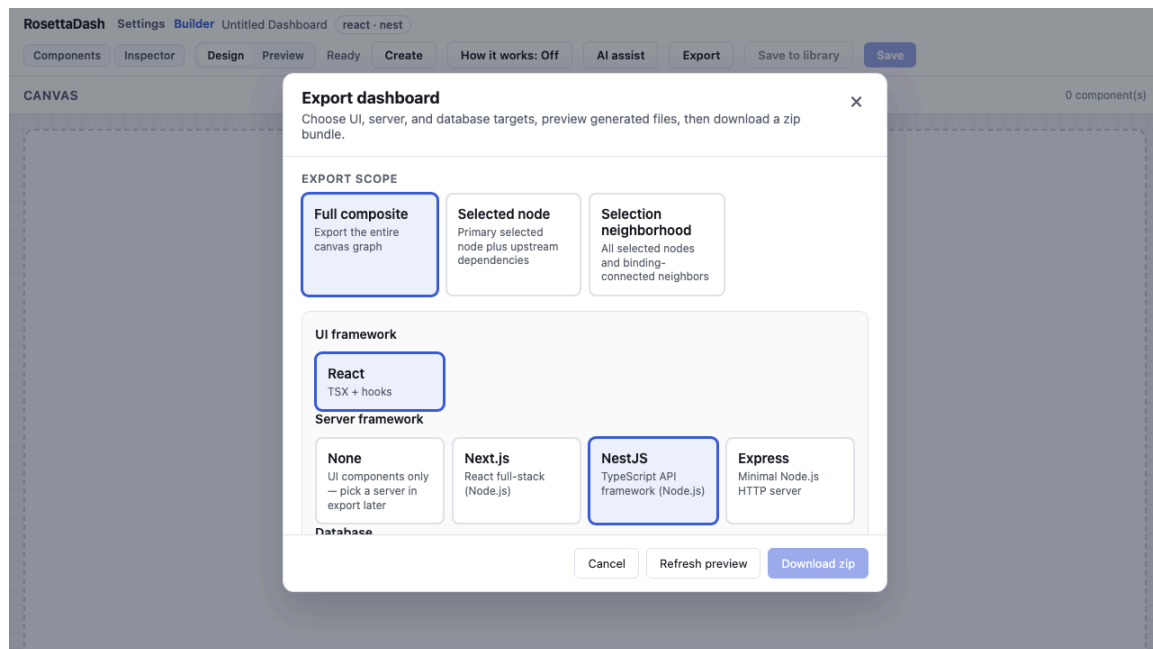


Figure 2: Export target picker — same canvas, choice of framework at export time.

Five proofs, one library

Destination Atlas ships five times — same thirty cities, same screen names, one shared library consumed by every proof app.

Runtime	Command	Port
Web Components	<code>npm run proof:web-components</code>	4310
React (reference UX)	<code>npm run proof:react</code>	4311
Angular	<code>npm run proof:angular</code>	4312
Vue	<code>npm run proof:vue</code>	4313
Svelte	<code>npm run proof:svelte</code>	4314

Open **About** in any proof. The matrix lists Package, Proof app, and Storybook; the current row, which references the framework being used, is marked “You are here.”

Destination Atlas
Current and historic information about world locations — Svelte proof (DAS-158)

Router: Browser (HTML5 history) | Role Viewer | App locale: English (English) | Map provider: Leaflet | Selected: Tokyo | Theme: System

About Destination Atlas

Destination Atlas is a functional demo — not a component kitchen sink. Each screen exercises real RosettaDash components the way you would wire them in production: filters bound to tables, maps with provider choice, role gates, and media embeds.

Why Destination Atlas?

The five proof apps (Web Components + four frameworks) share a 30-city mock library from `libs/destination-atlas` and identical screen names. Compare runtime imports side-by-side, then open Storybook on the matching port to inspect components in isolation.

Install `@rosettadash/core` and the runtime package you need (`@rosettadash/react`, `/angular`, `/vue`, `/svelte`, or `/web-components`). See `docs/39-npm-consumer-install.md` for tarball and registry workflows.

Runtimes — proof apps & Storybook

Each runtime ships a **proof app** (full Destination Atlas UX) and a **Storybook catalog** (isolated component review). Use the same npm package in your consumer project.

The Web Components, React, Angular, Vue, and Svelte blocks below are not RosettaDash palette components — they are npm runtime packages (delivery targets). Each card is proof-app documentation UI: plain layout markup styled for this About page. The actual reusable components live inside those packages (KpiCard, GeoMap, ScrollRegion, etc.) and appear on the other tabs and in Storybook.

PACKAGE	PROOF APP	STORYBOOK
Web Components (Custom Elements)		DAS-121
Shadow-DOM hosts and catalog elements. Map and Globe stay native rd-* custom elements; Authoring hosts the React subtree (DAS-159). Use when exporting CE-first dashboards or embedding rd-* tags in any stack.		
<code>@rosettadash/web-components</code>	<code>npm run proof:web-components</code> localhost:4310	<code>npm run storybook:web-components</code> localhost:6006

Figure 3: About runtime matrix — Package, Proof app, Storybook. You are here.

Mixing on purpose

As a simple POC, the **Svelte** proof purposefully hosts three framework guests:

- **Globe** — the Vue Three.js wrapper around the geo globe.
- **Media** — the Angular YouTube embed component.
- **Map** — the `<rd-geo-map>` custom element via `svelte:element`.

Authoring is native Svelte — `EquirectSphereViewport`, `FlatVideoViewport`, and `WasmMedia` from `@rosettadash/svelte` (DAS-179).

That is a realistic pattern: reuse what already works, wire it in one app frame, and document the bridge. Storybook showed each piece alone; Destination Atlas shows them together.

AboutOverviewDestinationsMapsMediaNewsAuthoringPlanStackSettings

Authoring

360° destination

Paris · 360°

Syncs with the header selection and autoloads the library clip when available. You can still click the source viewport to upload your own video.

360° SOURCE

Notre-Dame and Île de la Cité — DXR. Encoded from a Wikimedia Commons still.
[Commons file](#) · [Wikimedia Commons licensing](#)
License: CC BY-SA 3.0

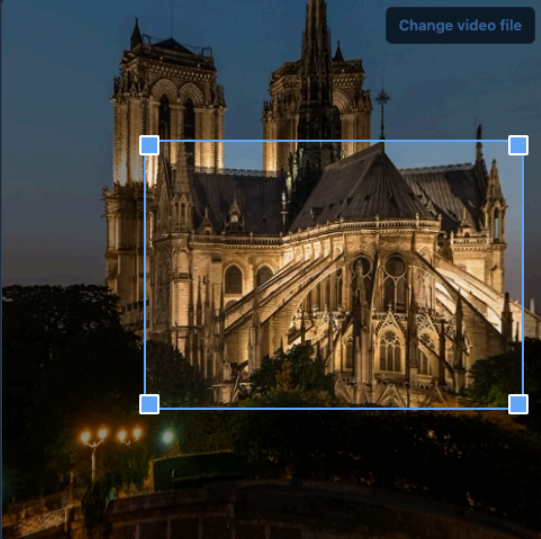
▼ 360° library — 25 shipped clips (JSON)

Autoload uses clipPath for the selected destination. Run `npm run authoring:fetch-360` to generate local MP4s.

```
[
  {
    "destinationId": "tokyo",
    "destinationName": "Tokyo",
    "sceneLabel": "Roppongi Hills",
    "credit": "nesnad",
    "license": "CC BY-SA 3.0",
    "commonsTitle": "360 panorama of Roppongi Hills - October 2016.jpg",
    "commonsUrl": "https://commons.wikimedia.org/wiki/File:360_panorama_of_Roppongi_Hills_-_October_2016.jpg",
    "clipPath": "/authoring-360/tokyo.mp4",
    "status": "shipped"
  },
  {
    "destinationId": "bangkok",
```

Source

Output



Change video file



Figure 4: Purposeful mix — Svelte shell with an embedded Vue Globe screen.

The smaller card demos do the same at widget scale: a React host page, a Vue host page, a custom-element 360° tour — each launched with an `npm run demo:*` script from the repo root.

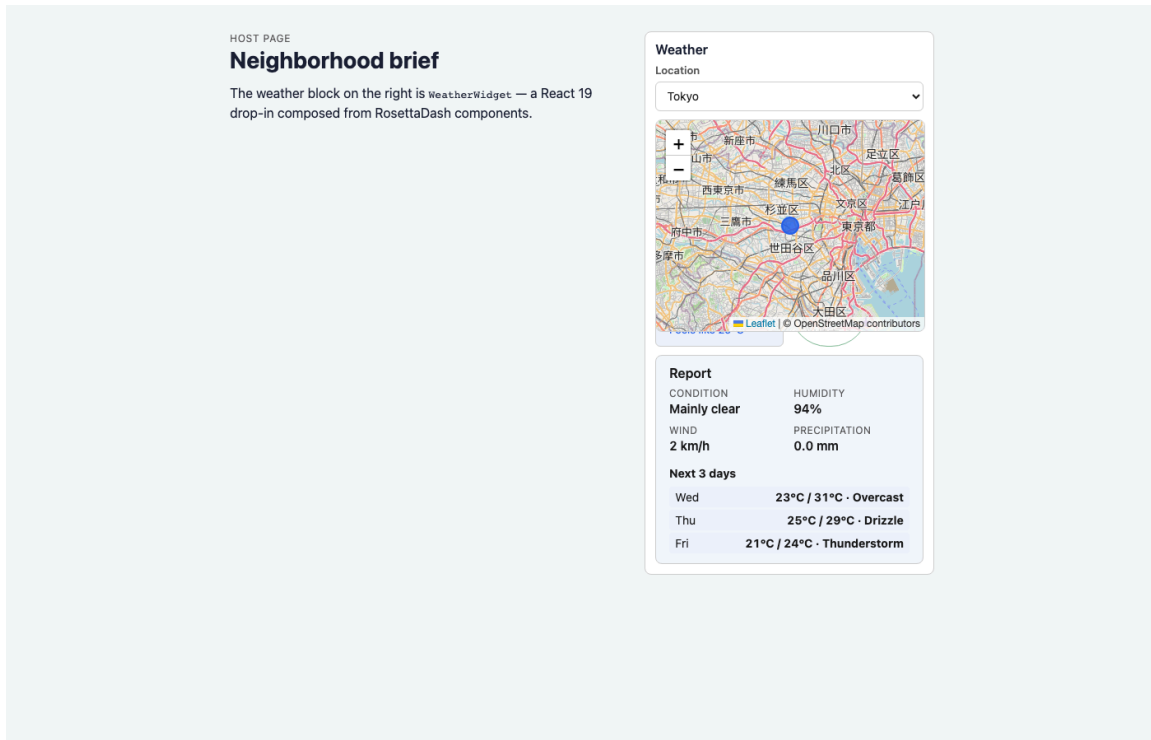


Figure 5: A demo widget on a host page — npm import at card scale.

Close

Clone the repo. Run the proof for the framework you use day to day, or run `npm start` and export a single KPI into a project you already have. Storybook is still there when you want one component in isolation.

The Full List of npm commands for launching apps

Run these from the cloned repo root. Each command starts a dev server; open the URL in your browser.

Builder and API

Command	URL
<code>npm start</code>	Builder UI — http://localhost:4200 NestJS API — http://localhost:3000/api Health check — http://localhost:3000/api/health
<code>npm run start:client</code>	Builder UI only — http://localhost:4200
<code>npm run start:server</code>	NestJS API only — http://localhost:3000/api

`npm start` runs client and server together. Use the split commands when you only need one side.

Backend parity stack (optional)

Proves exported server code against seeded databases in Docker.

Command	Purpose
<code>npm run parity:verify</code>	Full check — builder, Docker DBs, generate, servers, matrix
<code>npm run parity:stack:proof:react</code>	Live Stack demo — React proof → Next (:53103)
<code>npm run parity:stack:proof:angular</code>	Live Stack demo — Angular proof → Nest (:53101)
<code>npm run parity:stack:storybook:react</code>	Storybook full-stack orders → Next (:53103)
<code>npm run parity:stack:storybook:angular</code>	Storybook full-stack orders → Nest (:53101)
<code>npm run parity:generate:live</code>	Regenerate .parity/servers/ only (builder API up)
<code>npm run parity:down</code>	Stop parity containers

Low-level steps (`parity:db:up`, `parity:generate`, `parity:servers:up`) still exist when you need them individually. Live demos use the `:stack:*` scripts so you do not juggle terminals.

Live full-stack demos: React proof Stack → Next; Angular proof Stack → Nest; Storybook **Full-stack orders (live API)**. Same orders seed data in every database; generated servers on ports 53101–53104.

Destination Atlas proofs

Same product, five runtimes (ports 4310–4314):

Command	URL
npm run proof:web-components	http://localhost:4310
npm run proof:react	http://localhost:4311
npm run proof:angular	http://localhost:4312
npm run proof:vue	http://localhost:4313
npm run proof:svelte	http://localhost:4314

Storybook catalogs

One catalog per runtime (ports 6006–6010). All share the same sidebar; **Getting Started** → **Start here** is the landing story.

Command	URL
npm run storybook:web-components	http://localhost:6006
npm run storybook:react	http://localhost:6007
npm run storybook:vue	http://localhost:6008
npm run storybook:angular	http://localhost:6009
npm run storybook:svelte	http://localhost:6010
npm run storybook:all	All five URLs above (parallel)

Card demos

Standalone widget demos — eleven card-scale hosts (ports 4320–4331):

Command	Runtime	URL
npm run demo:weather	React	http://localhost:4320
npm run demo:news	React	http://localhost:4321
npm run demo:sensor	React	http://localhost:4322
npm run demo:stock	Angular	http://localhost:4323
npm run demo:sports	Angular	http://localhost:4324
npm run demo:crypto	Vue	http://localhost:4326
npm run demo:flight	Vue	http://localhost:4327
npm run demo:media	Svelte	http://localhost:4328
npm run demo:transit	Svelte	http://localhost:4329
npm run demo:tour	Web Components	http://localhost:4330
npm run demo:listing	Web Components	http://localhost:4331